

# Lineage-Guided Stale-Memory Repair for Reused Transformer KV Caches

A Systems Contract for Historical Retention, Operational Freshness, and Provenance

Anonymous Authors

Artifact path: `lineagekv`  
`paper/stale_memory_kv_cache_repair.tex`

May 21, 2026

## Abstract

Long-lived language-model agents increasingly reuse long-context key-value (KV) caches across turns to avoid expensive prefill. Reuse is efficient, but it also creates a systems failure mode: a cache can preserve obsolete memory records after a user’s preferences, task state, policy, or tool outcome has changed. We study *stale-memory cache repair*: suppressing obsolete facts inside a live transformer KV cache while preserving the full historical memory record. The method couples a memory-write protocol that records explicit lineage edges between superseded and current memories with post-prefill cache edits over the token spans of stale records. A provenance-repair layer then maps generated evidence aliases back to current memory identifiers. On a 32-case long-context 7B MLX benchmark, value repair matches or improves prompt deletion on answer and evidence metrics while avoiding full re-prefill, yielding a 15.35x persistent-update speedup at 6.7k tokens and 38.46x at approximately 32k tokens. On a raw downloadable Gemma 4 E4B-it checkpoint, a 32-case adversarial cross-domain matrix shows that full-cache reuse causes wrong or stale actions in 9/32 cases; prompt deletion prevents 9/9, value-only cache repair prevents 8/9, and key-only or K+V repair prevents 7/9, with no introduced wrong actions. The results support a practical systems claim rather than a universal mechanistic law: stale-cache repair is a real intervention class, but the best edit depends on model family, prompt interface, memory domain, and provenance requirements. The present evidence base is controlled and intentionally scoped; decisive journal claims require additional full-precision 7B+ GPU replication, non-synthetic traces, lineage-noise tests, and stronger cache-management baselines.

**Keywords:** transformer inference, KV cache, long-context agents, memory systems, retrieval, provenance, stale data, model editing, agent safety.

## 1 Introduction

Transformer inference is increasingly stateful. Production systems cache prefill activations, reuse context across turns, retrieve external memories, and ask a model to act on accumulated facts. This architecture makes agents faster and more useful, but it also creates a subtle mismatch between *historical memory* and *current operational state*. Historical records should often remain available for audit, temporal reasoning, and user trust. Operational decisions, however, should use the current record. If a system simply reuses a long-context KV cache that contains an obsolete memory, the model can act on stale content even when the memory store already knows that a newer record supersedes it.

The conventional fix is prompt deletion: remove obsolete records from the prompt and rebuild the prefill cache. Prompt deletion is simple and strong, but it has two drawbacks. First, it destroys the distinction between retaining history and using history for action. Second, in long contexts it forces a full prefill refresh after each relevant memory update. This is increasingly costly as context windows and agent memory stores grow.

This paper studies an alternative: *lineage-guided KV cache repair*. At memory-write time, the system records which memory records supersede earlier records. At runtime, the renderer maps memory records to token spans. After prefill, the inference runtime edits selected key and/or value vectors for token spans corresponding to stale memories. The full prompt and full cache remain present, but stale influence is suppressed. A second deterministic provenance layer resolves evidence identifiers from the memory lineage graph, so the model need not regenerate exact source IDs from memory.

The central scientific finding is not that a single edit is universally correct. The evidence points to a *phase diagram*. In Qwen2.5-style runs, value edits are often the cleanest intervention: values carry obsolete content, while keys preserve useful address structure. In Gemma 4 E4B-it, value-only, key-only, and K+V edits can all fix answer-level behavior on some domains, while provenance leakage remains domain-dependent. Prompt quality also matters: Gemma 4 E4B-it is clean on easy action-memory cases under its proper turn format, yet fails under harder adversarial memory conflicts. Thus, the deployable result is a two-layer contract: maintain memory lineage, choose a model/domain-appropriate cache policy, and repair provenance from the memory ledger.

**Contributions.** This manuscript makes five contributions:

1. We define stale-memory cache repair as a runtime systems problem for long-lived language-model agents that reuse long-context KV caches.
2. We introduce a lineage-guided cache-edit protocol that preserves historical memory records while suppressing stale token spans after prefill.
3. We provide a provenance-repair layer that decouples behavioral repair from exact citation generation.
4. We report a 7B long-context result with up to 38.46x persistent-update speedup and no loss in repaired joint answer/source correctness.
5. We report a raw Gemma 4 E4B-it 32-case cross-domain matrix showing wrong-action prevention by prompt deletion and cache repair, and we organize the results into a model/domain phase diagram.

## 2 Background and Related Work

**Transformer KV caches.** Autoregressive transformers cache attention keys and values to avoid recomputing past activations during decoding [1]. KV caches are also central to modern serving systems, where cache memory management strongly affects throughput and latency. PagedAttention and vLLM focus on efficient allocation and sharing of KV cache blocks for serving [5]; KV quantization work such as KIVI and KVQuant focuses on compressing caches while preserving quality [10, 11]. Our work is complementary: we do not primarily compress or page the cache; we edit semantically stale spans inside it.

**KV cache retention, eviction, and streaming.** H<sub>2</sub>O, Scissorhands, and StreamingLLM are the closest cache-management neighbors [7, 8, 9]. H<sub>2</sub>O keeps recent and heavy-hitter tokens, Scissorhands retains tokens predicted to remain important, and StreamingLLM preserves attention-sink tokens with a sliding window for stable streaming. These systems decide which KV entries to retain under memory or sequence-length constraints. Stale-memory repair has a different objective: a low-rank or low-salience token may still be current, and a high-attention token may be obsolete. The selection signal is memory lineage, not attention magnitude or recency. This distinction motivates direct comparisons to eviction and attention-mask baselines in future revisions.

**Prompt and prefix caching.** Prompt Cache and vLLM automatic prefix caching reuse previously computed attention states for shared prompt prefixes [6, 5]. Commercial prompt-caching APIs expose related serving semantics: users receive latency or cost benefits when long prompt prefixes are reused, while cache validity is generally tied to exact or near-exact prefix reuse rather than semantic freshness. Our setting asks what a serving system should do when the cached prefix is computationally reusable but operationally stale.

**Long-context and memory-augmented language models.** Retrieval-augmented generation and retrieval-enhanced transformers condition language models on external text rather than relying solely on parametric memory [3, 4]. Long-context transformers and segment recurrence methods extend the range of usable context [2]. Agent systems add persistent memories that can be queried across sessions; examples include OS-inspired memory orchestration in MemGPT/Letta [13]. These systems generally focus on what to retrieve, store, summarize, or page. We focus on what happens when retrieved or cached memory is obsolete but still historically valid.

**Model editing and activation intervention.** Model editing methods modify parameters or internal activations to change model facts. Stale-memory cache repair is narrower and more operational: the memory store already knows which record is current, and the objective is not to rewrite model weights but to repair a live cache state created by a particular prompt. This makes the intervention reversible, source-local, and compatible with normal inference.

**Provenance and source attribution.** Citation correctness is a separate requirement from answer correctness. In agentic systems, a stale action can occur even when the answer text looks plausible; conversely, a model can answer correctly while citing stale memory. We therefore measure answer accuracy, evidence accuracy, stale-use rate, action success, and repaired joint correctness. This separation is important throughout the results.

### 3 Problem Setting

Let a memory store contain records

$$m_i = (\text{id}_i, k_i, x_i, t_i, s_i, \mu_i),$$

where  $\text{id}_i$  is a source identifier,  $k_i$  is a memory key,  $x_i$  is text,  $t_i$  is a timestamp,  $s_i$  is a status, and  $\mu_i$  is metadata. A newer record  $m_j$  may supersede an older record  $m_i$ . We represent this by a directed lineage edge

$$m_i \rightarrow m_j,$$

meaning that  $m_i$  is historically retained but should not control current actions for the superseded key.

For a rendered prompt prefix  $P$ , let  $T_i \subset \{1, \dots, |P|\}$  be the token positions corresponding to memory record  $m_i$ . A prefill pass produces a per-layer KV cache

$$C = \{(K_\ell, V_\ell)\}_{\ell=1}^L.$$

The stale-memory repair task is to construct a repaired cache  $C'$  such that:

1. obsolete records remain present in the historical prompt and audit log,
2. stale token spans exert less influence on current answers and actions,
3. current records remain available for evidence and reasoning, and
4. the update path avoids full prompt re-prefill when possible.

**Policies.** We compare:

- **full\_cache**: prefill and decode from the full prompt containing current, stale, and distractor memories.
- **drop\_stale\_prompt**: remove stale records from the prompt before prefill.
- **value-only**: keep the full cache but edit selected stale value vectors.
- **key-only**: keep the full cache but edit selected stale key vectors.
- **K+V**: edit both stale key and stale value vectors.

Three natural baselines are not yet in the reported matrix and should be treated as required revisions before a strong journal claim: attention masking over stale prefix positions at decode time, span eviction with positional renumbering, and a current-only retrieval/RAG baseline that rebuilds the prompt from current memories only. We list them explicitly because they are closer competitors than **drop\_stale\_prompt** for some serving deployments, and the present manuscript should not be read as having ruled them out.

The concrete cache-editing policies used in the raw Gemma 4 E4B-it matrix edit lineage-conflict spans over layers 8–23. The 7B MLX long-context run uses a value policy over the same middle-layer band. This band was selected on development slices; a complete layer sweep is a required next validation. Some development scripts support current-span scaling, but the reported main rows use factor 1.0, so this manuscript does not claim a current-boost benefit.

## 4 Method

### 4.1 Memory Lineage Protocol

The memory protocol records supersession at write time. When a new memory is written to the same operational key, the old current memory is not deleted. It is marked superseded, and a lineage edge from old to new is stored. The renderer can then emit a full history prompt with neutral block aliases while retaining a private map from aliases to memory IDs.

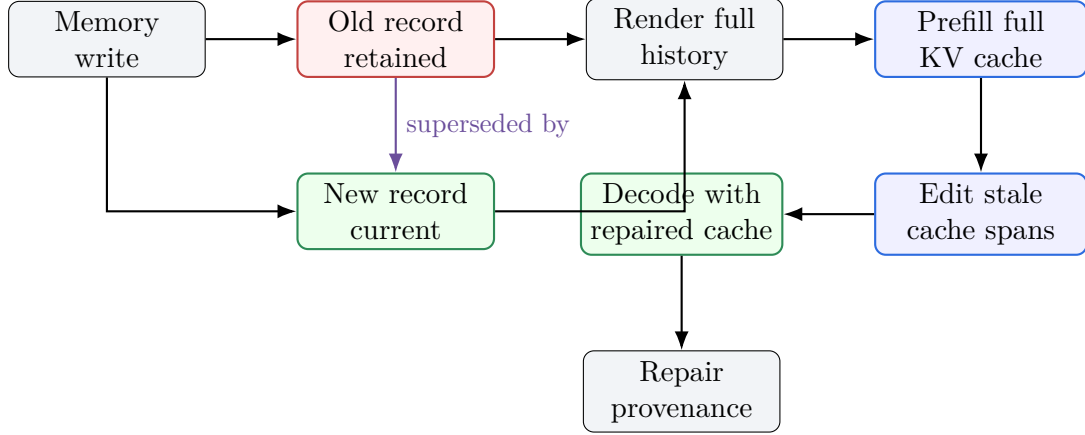


Figure 1: Lineage-guided cache repair. Historical records remain in memory and in the full prompt. Lineage identifies stale spans; cache repair suppresses their live influence after prefill; provenance repair maps citations back to current memory IDs.

## 4.2 Cache Editing

Let  $S$  be the union of stale token spans selected by lineage and rendering:

$$S = \bigcup_{m_i \rightarrow m_j} T_i.$$

Let  $\mathcal{L}$  be the selected layer set. A simple value-zeroing policy is:

$$V'_{\ell,p} = \begin{cases} 0, & \ell \in \mathcal{L}, p \in S, \\ V_{\ell,p}, & \text{otherwise.} \end{cases}$$

Key-only and K+V variants apply the analogous edit to  $K_{\ell,p}$  or to both  $K_{\ell,p}$  and  $V_{\ell,p}$ . The cache edit is local to the current inference run and does not alter model weights or delete memory records.

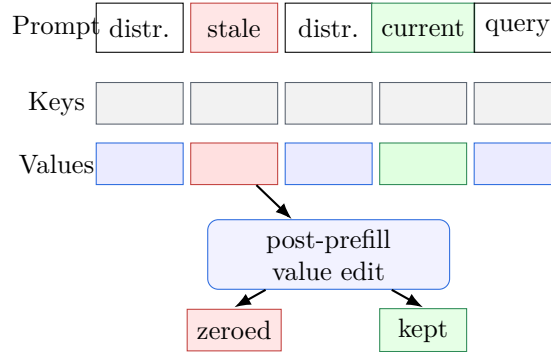


Figure 2: Schematic value-only repair. The full prompt remains present, but value vectors for lineage-stale token spans are suppressed after prefill. The production system can choose value-only, key-only, or K+V edits depending on the measured phase diagram.

## 4.3 Provenance Repair

Generated evidence IDs are fragile because a model may produce a neutral alias, a stale alias, or a malformed citation even when the answer text is correct. We therefore distinguish model-generated

evidence accuracy from repaired evidence accuracy. Provenance repair uses the same lineage graph that drives cache editing:

$$\text{repairEvidence}(m_i) = \begin{cases} m_j, & m_i \rightarrow m_j, \\ m_i, & \text{otherwise.} \end{cases}$$

This operation does not change the answer text. It provides a deterministic source contract for downstream systems that need current memory IDs.

## 5 Experimental Design

### 5.1 Datasets and Tasks

Table 1 summarizes the principal datasets and artifacts used in this manuscript. The benchmarks are generated assistant-memory and adversarial-lineage tasks plus public Git-history document-revision traces. We use generated tasks because they provide exact stale/current provenance and allow paired interventions over identical memory conflicts. This is also a limitation: the results should be interpreted as controlled systems evidence, not production telemetry.

| Dataset / artifact                     | Cases | Stale IDs | Purpose                                                                                                 |
|----------------------------------------|-------|-----------|---------------------------------------------------------------------------------------------------------|
| Public stale-memory benchmark v1       | 184   | 224       | Public assistant-memory benchmark covering nonce updates, action memory, and messy multi-update memory. |
| Universality trace benchmark           | 264   | 304       | Mixed benchmark: assistant-memory rows plus public Git-history document-revision rows.                  |
| MLX long-context action memory         | 32    | 32        | 7B long-context quality and persistent-update timing at about 6.7k tokens.                              |
| Exact-32k runtime slice                | 2     | 2         | Long-context externality check at approximately 32k tokens.                                             |
| Raw Gemma 4 E4B-it adversarial lineage | 32    | variable  | Cross-domain full/value/key/K+V matrix under proper Gemma turn formatting.                              |

Table 1: Datasets and artifacts used in the main evidence package. Stale IDs refer to records marked stale by lineage or text+timestamp currentness.

### 5.2 Models and Runtimes

The strongest systems result uses Qwen2.5-7B-Instruct-4bit in MLX. The raw-model award-track replication uses Gemma 4 E4B-it through Transformers CPU with bfloat16. The exact local Hugging Face model ID is `google/gemma-4-E4B-it`, snapshot `d6436b3d62967e1af08bbb046c6300b2a9ae8e85`. We also checked raw Qwen/Qwen3.5-9B cache compatibility at snapshot `c202236235762e1c871ad0ccb60c8ee5ba337b9a`; it exposes editable KV tensors only on a subset of hybrid-attention layers, and local CPU decoding is too slow for benchmark-scale answer-quality runs. Ollama behavioral checks are excluded from the mechanism claim because they do not expose editable KV caches.

### 5.3 Metrics

All quality metrics are computed by a deterministic grader over parsed JSON or the fallback answer string. The grader matches current/stale values and memory IDs from the benchmark ledger; it

does not use an LLM judge. We report:

- **Accuracy:** whether the answer contains the current gold value.
- **Evidence accuracy:** whether generated evidence IDs point to the gold current memory.
- **Stale-use rate:** whether the answer or evidence uses a stale value, stale evidence ID, or lineage-superseded alias.
- **Wrong-action rate:** one minus action success, where action success requires correctness and no stale use.
- **Repaired joint accuracy:** answer correctness together with provenance repaired to the current memory ID.
- **Persistent-update speedup:** prompt-deletion update time divided by cache-repair update time.

## 6 Main Results

### 6.1 Raw Gemma 4 E4B-it Cross-Domain Matrix

Table 2 gives the 32-case raw Gemma 4 E4B-it result. Full-cache reuse fails on both answers and provenance. Prompt deletion is a strong upper baseline. Cache repair closes the answer gap for all tested edit types, while value-only repair has the lowest stale-use rate among cache edits. Because the matrix has only 32 paired cases, one-row differences matter: the value-only advantage over key-only and K+V is a one-case stale-use margin, not a settled statistical ordering. A journal revision should add paired bootstrap intervals or McNemar tests to every cell and comparator.

| Policy            | Cases | Accuracy | Evidence | Stale use | Action success |
|-------------------|-------|----------|----------|-----------|----------------|
| full_cache        | 32    | 0.8125   | 0.6563   | 0.2813    | 0.7188         |
| drop_stale_prompt | 32    | 1.0000   | 1.0000   | 0.0000    | 1.0000         |
| value-only        | 32    | 1.0000   | 0.9375   | 0.0313    | 0.9688         |
| key-only          | 32    | 1.0000   | 0.9375   | 0.0625    | 0.9375         |
| K+V               | 32    | 1.0000   | 0.9375   | 0.0625    | 0.9375         |

Table 2: Raw Gemma 4 E4B-it adversarial-lineage matrix over seven domains. The metric triple used elsewhere is accuracy / evidence accuracy / stale-use rate.

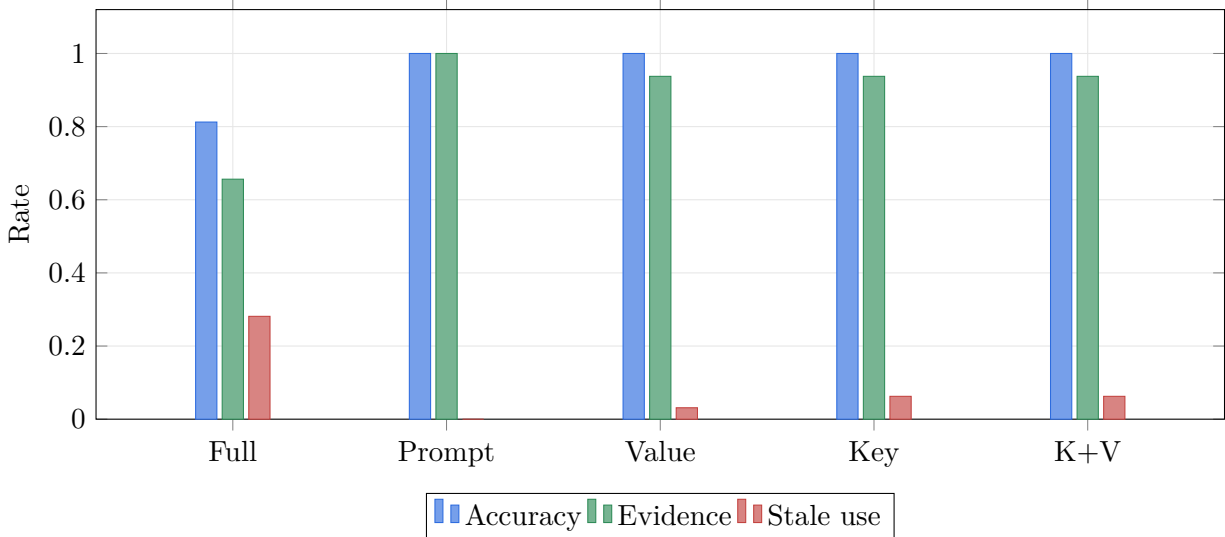


Figure 3: Raw Gemma 4 E4B-it 32-case policy matrix. Cache repair eliminates answer errors and strongly reduces stale use, but provenance leakage remains policy- and domain-dependent.

## 6.2 Wrong-Action Prevention

Table 3 shows paired wrong-action prevention relative to `full_cache`. The baseline has nine wrong/stale actions. Prompt deletion prevents all nine. Value-only cache repair prevents eight, while key-only and K+V prevent seven. No comparator introduces a new wrong action in this 32-case matrix.

| Comparator        | Baseline wrong | Prevented | Introduced | Prevention rate |
|-------------------|----------------|-----------|------------|-----------------|
| drop_stale_prompt | 9              | 9         | 0          | 1.000           |
| value-only        | 9              | 8         | 0          | 0.889           |
| key-only          | 9              | 7         | 0          | 0.778           |
| K+V               | 9              | 7         | 0          | 0.778           |

Table 3: Wrong-action prevention on the raw Gemma 4 E4B-it 32-case matrix. A wrong action is an incorrect or stale-contaminated answer.



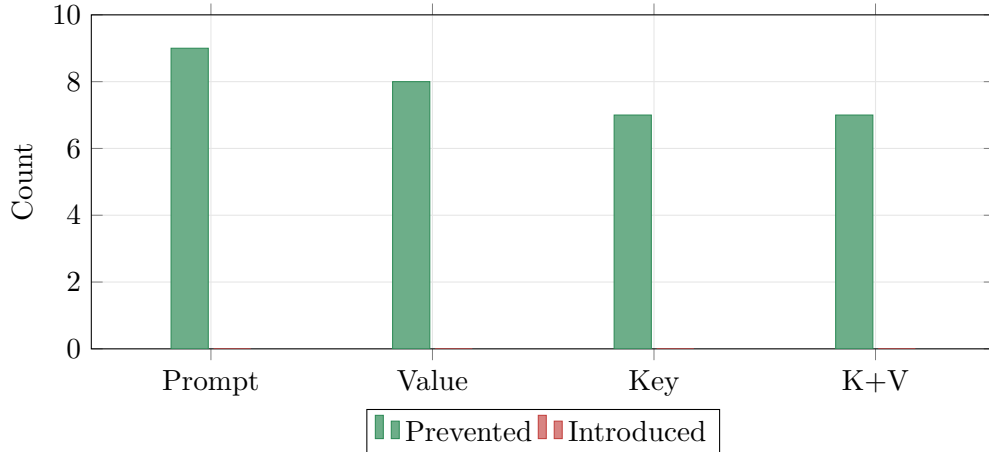


Figure 4: Paired wrong-action prevention relative to `full_cache` on raw Gemma 4 E4B-it.

### 6.3 Domain Phase Diagram

The domain breakdown in Table 4 is the main reason we avoid a universal-law claim. Some domains show answer/action failures; others show only provenance failures; one knowledge-base block is clean under proper prompting. The method is therefore best understood as a policy surface over model family, interface, domain, and provenance requirement. The positive stale-use signal is concentrated in five of the seven rows: deployment-state and knowledge-base have zero full-cache stale-use in this matrix and should be read as citation-precision or negative-boundary checks, not evidence for stale-action repair.

| Domain           | Cases | Full acc. | Full ev. | Full stale | Interpretation                                                                                 |
|------------------|-------|-----------|----------|------------|------------------------------------------------------------------------------------------------|
| User preference  | 8     | 0.625     | 0.625    | 0.375      | Stale user preference causes wrong actions; all cache edits are clean.                         |
| Policy approval  | 4     | 1.000     | 0.500    | 0.500      | Answer text survives, but stale evidence leaks; value-only is safest before provenance repair. |
| CRM follow-up    | 4     | 0.500     | 0.500    | 0.500      | Operational wrong-action domain; all cache edits are clean.                                    |
| Deployment state | 4     | 1.000     | 0.500    | 0.000      | Citation-precision domain; key and K+V repair recover exact evidence.                          |
| Tool trace       | 4     | 1.000     | 0.750    | 0.250      | Stale execution provenance appears despite correct answer.                                     |
| Task state       | 4     | 0.750     | 0.750    | 0.250      | Work-tracking stale state; all cache edits are clean.                                          |
| Knowledge base   | 4     | 1.000     | 1.000    | 0.000      | Clean under proper Gemma prompt; no repair upside visible.                                     |

Table 4: Raw Gemma 4 E4B-it domain phase diagram under the proper Gemma turn format. Columns report `full_cache` behavior.

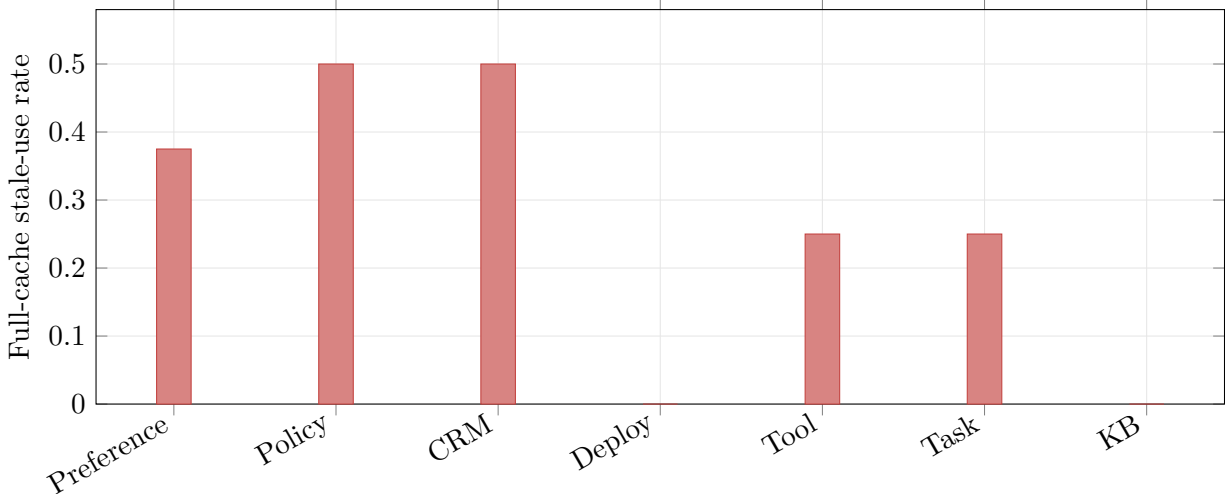


Figure 5: Full-cache stale-use rate varies by domain. This motivates model/domain-specific policy selection rather than one universal edit rule.

#### 6.4 Provenance Repair

Table 5 shows the effect of deterministic provenance repair on the raw Gemma 4 E4B-it matrix. Cache edits already recover answer accuracy, but model-generated evidence remains imperfect. Provenance repair restores all cache policies to 1.000 repaired joint correctness.

| Policy            | Accuracy | Original ev. | Original joint | Repaired ev. | Repaired joint |
|-------------------|----------|--------------|----------------|--------------|----------------|
| drop_stale_prompt | 1.0000   | 1.0000       | 1.0000         | 1.0000       | 1.0000         |
| full_cache        | 0.8125   | 0.6563       | 0.6563         | 1.0000       | 0.8125         |
| value-only        | 1.0000   | 0.9375       | 0.9375         | 1.0000       | 1.0000         |
| key-only          | 1.0000   | 0.9375       | 0.9375         | 1.0000       | 1.0000         |
| K+V               | 1.0000   | 0.9375       | 0.9375         | 1.0000       | 1.0000         |

Table 5: Provenance repair on raw Gemma 4 E4B-it. Deterministic lineage repair fixes source IDs, but it does not rescue wrong answers from `full_cache`.

#### 6.5 7B Long-Context Runtime Utility

The raw Gemma 4 E4B-it result establishes a second-family repair surface, but the strongest systems utility result comes from the MLX Qwen2.5-7B-Instruct-4bit long-context benchmark. Table 6 reports the quality result on 32 held-out long-context cases.

| Policy                  | Cases | Accuracy | Evidence | Stale use |
|-------------------------|-------|----------|----------|-----------|
| full_cache              | 32    | 1.0000   | 0.7500   | 0.2500    |
| drop_stale_prompt       | 32    | 1.0000   | 0.9375   | 0.0000    |
| Value repair (no boost) | 32    | 1.0000   | 0.9688   | 0.0313    |

Table 6: MLX Qwen2.5-7B-Instruct-4bit long-context 32-case result at approximately 6.7k tokens. With deterministic provenance repair, repaired evidence and repaired joint correctness reach 1.0000.

The paired bootstrap statistics for cache repair versus `full_cache` on this 7B run show an

evidence delta of +0.219 with confidence interval [0.094, 0.375], and a stale-use delta of -0.219 with confidence interval [-0.375, -0.094]. Answer accuracy is unchanged at 1.000.

## 6.6 Runtime Scaling

Table 7 and Figure 6 report the persistent update path. Prompt deletion requires a fresh shorter prefill. Cache repair reuses the full cache and edits stale spans. The speedup grows with context length: 15.35x at about 6.7k tokens and 38.46x at about 32k tokens. The ratio is expected to grow with context because prompt deletion repeats prefill while the edit path is closer to span-local; the primary measurement is therefore the absolute update time in milliseconds.

| Context scale | Cases | Prompt deletion<br>update ms | Cache repair<br>update ms | Speedup |
|---------------|-------|------------------------------|---------------------------|---------|
| 6.7k d48/s4   | 32    | 19,380.0                     | 1,262.3                   | 15.35x  |
| 31k d230/s4   | 4     | 85,253.8                     | 2,247.4                   | 37.93x  |
| 32k d236/s4   | 2     | 119,077.0                    | 3,096.5                   | 38.46x  |

Table 7: Persistent-update utility for MLX Qwen2.5-7B-Instruct-4bit.

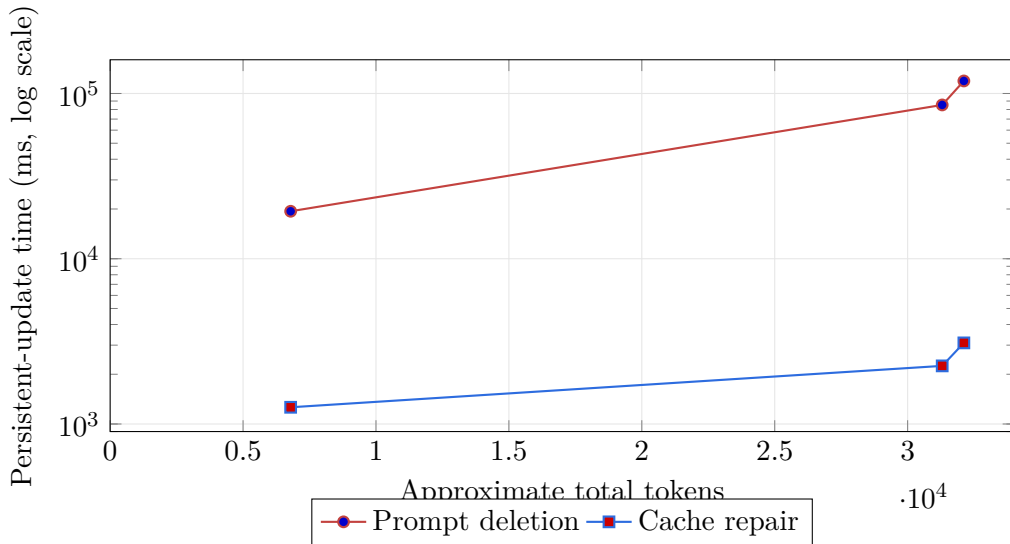


Figure 6: Absolute persistent-update time. Cache repair avoids full re-prefill; the speedup ratios in Table 7 are derived from these paired measurements.

## 7 Ablations and Boundary Results

### 7.1 Address/Content Dissociation in Qwen2.5

In smaller Qwen2.5 runs, value-only repair is substantially better than key-only repair. A representative causal-map result is shown in Table 8. We treat this as an exploratory address/content hypothesis for this model family, not as a proven mechanism. The key-only collapse could reflect useful-routing damage, but it could also reflect more mechanical effects such as attention-norm disruption, rotary position interactions, or distribution shift from hard zeroing. A stronger mechanistic

section would compare zeroing with mean replacement, norm-matched noise, and attention-weight redistribution probes.

| Model        | Edit       | Accuracy | Evidence | Stale use |
|--------------|------------|----------|----------|-----------|
| Qwen2.5-0.5B | full cache | 0.362    | 0.450    | 0.650     |
| Qwen2.5-0.5B | value-only | 0.412    | 0.650    | 0.375     |
| Qwen2.5-0.5B | key-only   | 0.087    | 0.000    | 0.000     |
| Qwen2.5-0.5B | K+V        | 0.287    | 0.225    | 0.263     |
| Qwen2.5-1.5B | full cache | 0.375    | 0.000    | 1.000     |
| Qwen2.5-1.5B | value-only | 0.425    | 0.950    | 0.425     |
| Qwen2.5-1.5B | key-only   | 0.150    | 0.025    | 0.175     |
| Qwen2.5-1.5B | K+V        | 0.300    | 0.500    | 0.175     |

Table 8: Representative Qwen2.5 exploratory ablation. Unlike the raw Gemma 4 E4B-it matrix, Qwen2.5 shows a clearer value-only advantage over key editing, but the mechanistic interpretation remains provisional.

## 7.2 Prompt Interface Boundary

The raw Gemma 4 E4B-it experiments reveal a prompt-interface boundary. Under a plain prompt, a four-case action-memory smoke shows strong stale-cache failures and malformed outputs. Under a minimal text-only implementation of the Gemma 4 turn format, the easy action-memory slice becomes clean: `full_cache`, `drop_stale_prompt`, and K+V repair all reach 1.000 accuracy / 1.000 evidence / 0 stale use over eight current-first and stale-first cases. Harder adversarial lineage cases are still failure-inducing. This distinction matters for scientific interpretation: repair results should not be attributed to cache mechanism when the prompt interface itself is invalid.

## 7.3 Preliminary Raw Qwen3.5-9B Smoke

After the initial compatibility check, we added a no-thinking chat-template path for `Qwen/Qwen3.5-9B` and ran a small CPU-only behavioral smoke on a no-distractor adversarial-lineage user-preference slice. This is not the full-precision CUDA replication requested in Section 9: local MPS fails with an incompatible-dimensions error, and CPU decoding through the torch fallback is too slow for a 32-case matrix. It is nevertheless useful boundary evidence because the raw checkpoint now produces parseable answers and the cache edit changes stale behavior.

| Policy                         | Cases | Accuracy | Evidence | Stale use |
|--------------------------------|-------|----------|----------|-----------|
| <code>full_cache</code>        | 4     | 0.250    | 0.000    | 0.750     |
| <code>drop_stale_prompt</code> | 4     | 1.000    | 0.750    | 0.000     |
| Value repair, layers 8–23      | 4     | 1.000    | 0.750    | 0.000     |

Table 9: Preliminary `Qwen/Qwen3.5-9B` CPU smoke on a no-distractor adversarial lineage slice. Prompt deletion and value repair both prevent 3/3 full-cache wrong actions with no introduced wrong actions. Evidence is 0.750 because one repeated-current case cites earlier current duplicates rather than the latest current duplicate; no comparator cites stale evidence.

## 7.4 Targeting Validity

The final deployment contract does not require hidden stale labels. The public 184-case benchmark has exact lineage and text+timestamp targeting: 224 true positive stale records, 0 false positives, and 0 false negatives. The mixed 264-row universality trace benchmark is also exact for both lineage and text+timestamp detectors across 304 stale records. These audits establish that the cache edit plan can be generated from programmatic currentness rather than from model reasoning over stale labels.

## 8 Discussion

### 8.1 What Is the Discovery?

In plain terms, the discovery is that stale memories inside a reused transformer cache can be repaired without deleting the historical record or rebuilding the whole prompt. The system needs to know which memory superseded which older memory. Once it has that lineage, it can target the old record’s token spans in the KV cache and suppress their live effect. The old memory still exists for audit and temporal context, but it stops controlling current actions.

### 8.2 Why Not Just Replace Memories?

Replacing or deleting memories is sufficient when history is irrelevant and contexts are short. Long-lived agents need a stricter contract. They should remember that a meeting used to be in Room 3B, that a policy used to require approval, or that a tool call previously failed. Those facts can matter for explanations, audits, and temporal reasoning. The problem is not historical retention; it is stale operational influence. Cache repair separates these concerns.

### 8.3 Why a Phase Diagram?

The results do not justify saying that stale memory always lives in values or that a fixed layer band is universal. Qwen2.5 shows a strong value/content dissociation. Raw Gemma 4 E4B-it shows that value-only, key-only, and K+V edits can all fix answers in some domains, while evidence leakage differs by domain. The right practical abstraction is a phase diagram: choose a repair policy after measuring the model/runtime/domain regime.

### 8.4 Real-World Impact

The immediate utility is for agents with persistent memory and expensive long-context prefill. Examples include coding agents, support copilots, CRM assistants, research assistants, and workflow agents that maintain user or task state. Consider a support copilot with a cached account history: an old memory says a customer is approved for an immediate refund, while a later policy record says refunds over a threshold require fraud-review approval. The old record should remain auditable because it explains earlier correspondence, but it should not authorize the current action. Prompt deletion can solve this at the cost of rebuilding the prefill. Cache repair gives a path to preserve the full cached history while suppressing obsolete operational influence.

## 9 Limitations

1. The strongest long-context runtime result uses MLX 4-bit Qwen2.5, not full-precision CUDA. A single full-precision 7B+ GPU replication is the highest-value next experiment.
2. The raw Gemma 4 E4B-it matrix is one raw non-Qwen model family on local CPU. It is benchmark-scale for this project, but not a broad model zoo.
3. The principal benchmarks are generated or controlled. They provide exact lineage and paired interventions, but they are not production telemetry. The public Git-history rows provide a partial non-synthetic anchor, but the main 7B and raw-Gemma 4 E4B-it headline tables are generated.
4. The current main tables lack attention-mask, span-eviction, and current-only retrieval baselines. These are required before claiming that KV editing is the best available serving strategy.
5. Statistical reporting is incomplete for the raw Gemma 4 E4B-it matrix. The next draft should include paired bootstrap intervals or exact paired tests for every main cell and comparator.
6. The layer band 8–23 is development-selected. We report no layer-band sweep or nontrivial  $\alpha$  sweep in the main 7B/Gemma 4 E4B-it results; all current-boost claims should be withheld until that sweep is run.
7. The memory ledger integration is demonstrated, but a large deployed agent study remains future work.
8. Provenance repair assumes that the memory store has reliable lineage or currentness metadata. We have not yet quantified robustness to missing, flipped, or noisy lineage edges. Without reliable lineage, the system should fail closed.
9. Cache edit policies are runtime- and architecture-dependent. Hybrid attention models may expose only a subset of editable KV layers.

## 10 Ethics and Safety

Stale-memory repair is a safety mechanism for reducing obsolete instructions and stale facts in agent decisions. It can also be misused if memory lineage is incorrectly recorded or if an operator hides historical records while claiming they are preserved. We therefore recommend auditable memory ledgers, explicit currentness metadata, conservative fail-closed behavior, and evaluation that separates answer correctness from provenance correctness. The generated benchmarks in this work do not contain private user data.

## 11 Reproducibility and Artifact Availability

The repository contains scripts, datasets, result CSVs, and an interactive demo. Key artifacts include:

- `data/public/stale_memory_repair_benchmark_v1.jsonl`
- `data/public/stale_memory_universality_traces_v1.jsonl`

- `scripts/kv_value_ablation_probe.py`
- `scripts/kv_mlx_value_ablation_probe.py`
- `src/context_rot/memory/ledger.py`
- raw Gemma 4 E4B-it driver: `scripts/kv_value_ablation_probe.py`  
`-model google/gemma-4-E4B-it`  
`-prompt-style gemma4`
- `results/kv/kv_gemma4_e4b_adversarial_lineage_crossdomain32_gemma4prompt_matrix_summary.csv`
- `results/kv/kv_gemma4_e4b_raw_cache_compat.json`
- `results/kv/kv_qwen35_9b_raw_cache_compat.json`
- `results/kv/kv_qwen35_9b_adversarial_lineage_d0_probe4_nothink_cpu_summary.csv`
- `results/kv/kv_public_runtime_curve.csv`
- `demo/kv_memory_repair_demo.html`

The no-model public reproduction entry point is:

```
bash scripts/run_kv_public_repro.sh
```

## 12 Conclusion

Reused long-context KV caches create a stale-memory failure mode for long-lived agents. We show that memory lineage can make stale spans targetable, that cache edits can suppress stale influence without deleting historical memory, and that deterministic provenance repair can restore current source IDs. The strongest systems result shows large update-path speedups at long context. The raw Gemma 4 E4B-it matrix shows that the repair class transfers beyond the original Qwen2.5 setting, while also revealing domain and prompt-interface boundaries. The scientific contribution is therefore a deployable systems contract and a model/domain phase diagram, not a universal transformer law. The next decisive step is independent raw 7B+ or full-precision CUDA replication and evaluation on realistic production-like memory logs.

## A Algorithmic Summary

| Step | Operation                                                                                            |
|------|------------------------------------------------------------------------------------------------------|
| 1    | Write memory record with key, timestamp, text, status, and source ID.                                |
| 2    | If the key has an earlier current record, mark it superseded and add a lineage edge from old to new. |
| 3    | Render full memory history with neutral aliases and record token spans for every memory record.      |
| 4    | Prefill the model on the full rendered prompt.                                                       |
| 5    | Build stale token span set from lineage edges and render spans.                                      |
| 6    | Apply selected cache edit over selected layers: value-only, key-only, or K+V.                        |
| 7    | Decode from the repaired cache.                                                                      |
| 8    | Repair or verify evidence IDs using the lineage graph before downstream action.                      |

Table 10: Lineage-guided stale-memory cache repair algorithm.

## B Raw Gemma 4 E4B-it Artifact Map

| Result                                  | Artifact path                                                                                                |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 32-case matrix summary                  | results/kv/kv_gemma4_e4b_adversarial_lineage_crossdomain32_gemma4prompt_matrix_summary.csv                   |
| 32-case wrong-action summary            | results/kv/kv_gemma4_e4b_adversarial_lineage_crossdomain32_gemma4prompt_matrix_wrong_action_summary.csv      |
| 32-case provenance repair summary       | results/kv/kv_gemma4_e4b_adversarial_lineage_crossdomain32_gemma4prompt_matrix_provenance_repair_summary.csv |
| Raw Gemma 4 E4B-it cache compatibility  | results/kv/kv_gemma4_e4b_raw_cache_compat.json                                                               |
| Raw Qwen/Qwen3.5-9B cache compatibility | results/kv/kv_qwen35_9b_raw_cache_compat.json                                                                |
| Task-state rows                         | results/kv/kv_gemma4_e4b_adversarial_lineage_task_state_offset100_4_gemma4prompt_matrix_cpu.jsonl            |
| Knowledge-base rows                     | results/kv/kv_gemma4_e4b_adversarial_lineage_knowledge_base_offset120_4_gemma4prompt_matrix_cpu.jsonl        |

Table 11: Raw Gemma 4 E4B-it artifacts created for the 32-case matrix.

## References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. arXiv:1706.03762.
- [2] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of ACL*, 2019.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of ICML*, 2022. arXiv:2112.04426.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, 2023.
- [6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt Cache: Modular attention reuse for low-latency inference. In *Proceedings of MLSys*, 2024. arXiv:2311.04934.
- [7] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H<sub>2</sub>O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.14048.



- [8] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.17118.
- [9] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. arXiv:2309.17453, 2023.
- [10] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of ICML*, 2024. arXiv:2402.02750.
- [11] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. arXiv:2401.18079, 2024.
- [12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [13] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.